Lista de Exercícios 3 - Introdução a Objetos no C#

Prof. Dr. Rodrigo Colnago Contreras
contreras@unifesp.br
Lattes Google Scholar ORCID

Criada em 28/09/2025 e modificada em 21/04/2026.

Exercícios

Exercício 1. Implemente as classes a seguir utilizando os conceitos de Programação Orientada a Objetos:

(a) Crie uma classe `Pessoa` com os atributos privados `nome` e `idade`. Implemente métodos de acesso (*getters*) e modificação (*setters*). No *setter* de idade, garanta que apenas valores positivos sejam aceitos.

(b) Crie uma classe `Aluno`, que herda de `Pessoa`, adicionando o atributo `nota`. Implemente os métodos de acesso e modificação, garantindo que a nota esteja entre 0 e 10.

(c) Utilize *polimorfismo* para sobrescrever um método `exibir_dados()`, de modo que:

- na classe `Pessoa`, exiba nome e idade;
- na classe `Aluno`, exiba nome, idade e nota.

Exercício 2. Utilize os conceitos de POO abordados em aula para resolver os seguintes itens:

(a) Construa uma classe `Numero_Natural` com apenas um atributo protegido `n` que deve ser um valor inteiro não negativo. Garanta, por meio do método de configuração `set_n`, que apenas valores inteiros positivos serão atribuídos a `n`. O construtor desta classe deve receber um parâmetro `n` que pode ser tanto um valor inteiro positivo quanto um *valor literal* que represente um inteiro positivo. No segundo caso, converta o literal para inteiro antes de configurar `n`.

(b) Construa uma classe `Numero_Inteiro`, herdeira de `Numero_Natural`. Usando *polimorfismo*, ajuste o método `set_n` para aceitar qualquer número inteiro (positivo, negativo ou zero).

(c) Construa uma classe `Numero_Racional` definida por dois atributos privados, ambos objetos da classe `Numero_Inteiro`: `numerador` e `denominador`. Utilize encapsulamento para definir métodos de acesso e configuração. O atributo `denominador` não pode assumir valor nulo; implemente a validação no respectivo *setter*.

Exercício 3. Modifique a classe `Numero_Racional` definida no exercício anterior acrescentando:

(a) Um método `show` que apresente na tela os atributos `numerador` e `denominador` no formato `numerador / denominador`.

(b) Sobrecarga do operador lógico de igualdade `==` para decidir se dois objetos de `Numero_Racional` são iguais.

(c) Sobrecarga do operador de soma `+` para permitir somar dois objetos `Numero_Racional`. Especificamente, se r_1 e r_2 têm numeradores a_1, a_2 e denominadores b_1, b_2, então $r_3 = r_1 + r_2$ deve ter numerador $a_1b_2 + a_2b_1$ e denominador b_1b_2.

Exercício 4. Crie uma classe `Carta` com três atributos privados: `valor`, `naipe` e `peso`. Utilize encapsulamento para acessar e alterar os valores, observando:

- `valor` ∈ `[A,2,3,4,5,6,7,8,9,10,J,Q,K]`;
- `naipe` ∈ `[Paus, Copas, Espadas, Ouros]`;
- `peso` é um número real positivo.

Sobrecaregue os operadores `==`, `>` e `<` para comparar duas `Cartas`. Uma carta é maior que outra se o seu `peso` for maior.

Exercício 5. Crie uma classe `Baralho` com os seguintes atributos:

- `valores_validos`: lista de valores possíveis de uma carta;
- `naipes_validos`: lista de naipes possíveis;
- `cartas`: lista de objetos `Carta` do baralho.

A classe deve seguir as convenções de encapsulamento e incluir os métodos:

- `constroi_baralho`: confecciona todas as cartas possíveis a partir de `valores_validos` e `naipes_validos`;
- `embaralha`: embaralha (aleatoriza) os índices de `cartas`;
- `mostra_cartas`: mostra todas as cartas em `cartas`;
- `retira_carta_aleatoria`: retira e retorna uma carta aleatória de `cartas`.

Exercício 6. Usando a classe `Baralho` do exercício anterior, crie três objetos: um *baralho italiano*, um *baralho inglês* e um *baralho inglês duplo*. Mostre todas as cartas de cada baralho com `mostra_cartas`. Os `pesos` das cartas devem ser inteiros positivos referentes aos seus valores; casos especiais: `A`=1, `J`=11, `Q`=12, `K`=13.

Obs.: Todos os baralhos têm os mesmos naipes (`Paus`, `Copas`, `Espadas`, `Ouros`). No *baralho italiano*, os valores 8, 9 e 10 não são válidos (use `[A,2,3,4,5,6,7,J,Q,K]`). No *baralho inglês*, use todos os valores `[A,2,3,4,5,6,7,8,9,10,J,Q,K]`. No *baralho inglês duplo*, considere dois baralhos ingleses (valores duplicados).

Exercício 7. Crie uma classe `Baralho_de_Truco`, herdeira de `Baralho` (do exercício 4), representando um *baralho italiano* com os seguintes pesos:

Carta	Peso
4	0
5	1
6	2
7	3
Q	4
J	5
K	6
A	7
2	8
3	9
Manilha de ouro (Ourilha)	10
Manilha de espadas (Espadilha)	11
Manilha de copas (Copilha)	12
Manilha de paus (Zap)	13

Acrescente:

- **Atributo** `vira`: uma `Carta` virada na mesa.
- **Método** `define_vira`: escolhe aleatoriamente uma carta do baralho e a define como `vira`.
- **Método** `atualiza_pesos`: após `define_vira`, atualiza os pesos das *manilhas* (vedetes). Se a `vira` for a carta de peso p, as manilhas são as cartas de peso $p + 1$, exceto quando a `vira` for um 3, caso em que as manilhas são as cartas de valor 4. Ex.: se virar um Ás, as manilhas são as cartas de valor 2 (2 de ouros, 2 de espadas, 2 de copas, 2 de paus), que passam a ter os pesos das manilhas correspondentes.

Por fim, defina um objeto desta classe e apresente as cartas e seus pesos *antes* e *depois* da execução de `define_vira`.